BÁO CÁO BÀI THỰC HÀNH SỐ 2
THU THẬP, LƯU TRỮ VÀ TIỀN XỬ LÝ DỮ LIỆU IoT

Học phần: IoT và Ứng dụng (INT14149)

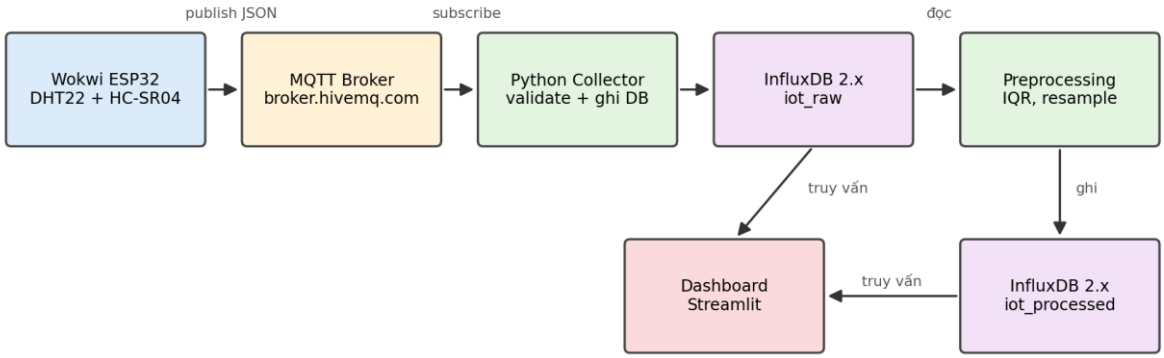
Thông tin	Nội dung
Họ và tên	Hoàng Minh Trọng
Mã sinh viên	B23DCCN847
Lớp và nhóm thực hành	IoT-05-02
Ngày lập và rà soát báo cáo	18/10/2026
Mã nguồn GitHub	[Bạn điền link GitHub]
Dự án Wokwi Bài 2	[Bạn điền link Wokwi]

1. Mục tiêu và mối liên hệ với Bài 1

Bài 1 xây dựng nút IoT ảo gồm ESP32, cảm biến nhiệt độ – độ ẩm DHT22, cảm biến khoảng cách HC-SR04 và LED trạng thái trên Wokwi, gửi telemetry bằng MQTT lên ThingsBoard và hiển thị bằng dashboard của nền tảng. Bài 2 phát triển tiếp từ chính nút cảm biến đó: giữ nguyên mạch, chân kết nối và các khóa dữ liệu, nhưng tự xây dựng toàn bộ chuỗi phía sau gồm thu thập có kiểm tra chất lượng, lưu trữ chuỗi thời gian, tiền xử lý và dashboard giám sát, nhằm hiểu rõ những việc mà nền tảng ThingsBoard đã làm sẵn trong Bài 1.

Nội dung	Bài 1	Bài 2 (phát triển)
Mạch Wokwi	ESP32 DevKit V1; DHT22 (GPIO15); HC-SR04 (GPIO5/18); LED (GPIO2)	Giữ nguyên diagram.json và libraries.txt
Telemetry	temperature, humidity, distance_cm, rssi, sequence, uptime_s	Giữ nguyên, thêm device_id và ts (NTP, ms)
Chu kỳ gửi	5 giây	2 giây để có nhiều mẫu cho tiền xử lý
Đích dữ liệu	ThingsBoard, topic v1/devices/me/telemetry	Broker MQTT, topic iot/<site>/<device>/telemetry
Lưu trữ	Do ThingsBoard đảm nhiệm	InfluxDB 2.x, thiết kế schema và retention riêng
Kiểm tra dữ liệu	Loại NaN của DHT trên ESP32	Thêm validate, loại trùng, phát hiện mất gói, outlier ở collector và tiền xử lý
Hiển thị	Dashboard ThingsBoard	Dashboard Streamlit: dữ liệu thô, đã xử lý, độ trễ

2. Kiến trúc hệ thống



Hình 1. Kiến trúc pipeline thu thập – lưu trữ – tiền xử lý – giám sát

Vì ESP32 chạy trong Wokwi không truy cập được máy chủ nội bộ trên máy tính, MQTT broker phải nằm trên Internet (HiveMQ). Collector chạy trên máy cá nhân và ghi vào InfluxDB OSS 2.10.0 chạy trực tiếp bằng influxd.exe trên Windows, không cần Docker. Cấu trúc topic iot/<site>/<device>/telemetry cho phép thu dữ liệu nhiều thiết bị; site và device_id được tách từ topic làm tag. Firmware đăng trạng thái retained và cấu hình Last Will để broker báo offline khi mất kết nối.

Bản tin mẫu: `{"device_id":"esp32-01","ts":1760000000000,"sequence":12,"temperature":25.6,"humidity":50.0,"distance_cm":10.02,"rssi":-87,"uptime_s":24}`

Khi HC-SR04 không nhận được xung phản hồi hoặc khoảng cách ngoài miền 2–400 cm, ESP32 bỏ khóa distance_cm thay vì gửi giá trị -1 như Bài 1, để dữ liệu thiếu không bị nhầm với một giá trị đo hợp lệ.

3. Thiết kế schema cơ sở dữ liệu

Measurement	Tags	Fields	Bucket / retention
sensor_raw	device_id, location	temperature, humidity, distance_cm, rssi, sequence, uptime_s, latency_ms	iot_raw / 30 ngày
collector_stats	collector	write_ms, batch	iot_raw / 30 ngày
sensor_processed	device_id	giá trị đã xử lý và các hậu tố _outlier, _roll, _delta, _z, _minmax của 3 đại lượng; n_raw	iot_processed / 90 ngày

Tag chỉ gồm các thuộc tính ít giá trị (thiết bị, địa điểm) để giữ series cardinality thấp; các giá trị đo, rssi, sequence và uptime_s là field. Timestamp của điểm là thời gian trên thiết bị nên khi một bản tin bị gửi lặp, điểm mới ghi đè điểm cũ cùng series và timestamp (idempotent). Retention gắn với bucket: dữ liệu thô giữ 30 ngày, dữ liệu đã resample nhỏ hơn nhiều nên giữ 90 ngày.

4. Cài đặt chính

4.1. Firmware và collector

- Firmware phát triển từ sketch Bài 1: đọc DHT22 (GPIO15) và HC-SR04 (distance = duration × 0,0343 / 2), thêm timestamp NTP, device_id, Last Will, tự kết nối lại Wi-Fi và MQTT; tăng bộ đệm PubSubClient lên 512 byte vì payload dài hơn mặc định 256 byte.
- Validate ở collector: JSON hợp lệ, đủ trường, đúng kiểu số, trong giới hạn vật lý (−40Chưa đo80 °C, 0Chưa đo100 %RH, 2Chưa đo400 cm). Gói sai bị loại và ghi log.
- Trùng lặp: bỏ gói có cùng (device, sequence, ts). Mất gói: phát hiện khoảng nhảy của sequence. Khởi động lại: uptime_s giảm cùng sequence về nhỏ được ghi nhận là restart chứ không tính là trùng hay mất gói, đúng với vai trò của hai trường này đã phân tích ở Bài 1.
- Lỗi cơ sở dữ liệu: điểm dữ liệu được giữ trong buffer (tối đa 5000) và ghi lại ở lần sau. Collector đo latency_ms (thời điểm nhận trừ ts thiết bị) và write_ms (thời gian ghi).

4.2. Tiền xử lý

Bước	Thao tác	Chi tiết
1	Đọc dữ liệu	Flux theo khoảng thời gian, pivot field thành cột, bỏ timestamp trùng
2	Phát hiện outlier	IQR (k = 1,5) hoặc Z-score (|z| > 3) cho từng đại lượng; outlier đặt thành NaN và đếm
3	Resample	Cửa sổ 10 s, lấy trung bình mẫu sạch; ghi n_raw và số outlier mỗi cửa sổ
4	Giá trị thiếu	Nội suy theo thời gian tối đa 3 cửa sổ; khoảng trống dài hơn giữ trống
5	Đặc trưng	Rolling mean 6 cửa sổ (60 s), delta giữa các cửa sổ
6	Chuẩn hóa	Z-score và Min-Max [0,1]
7	Lưu	Measurement sensor_processed trong bucket iot_processed

5. Kết quả thực nghiệm

Đã chạy InfluxDB OSS 2.10.0 cục bộ với bucket iot_raw (30 ngày) và iot_processed (90 ngày), rồi kiểm chứng luồng simulator Python → MQTT HiveMQ → collector → InfluxDB trong khoảng 80 giây. Luồng này dùng simulator, không phải Wokwi. Sau đó cài Python 3.12.15 64-bit portable và requirements.txt vào môi trường .venv-dashboard; dashboard Streamlit chạy tại http://127.0.0.1:8501 và truy vấn được 38 điểm raw, 9 điểm processed. Phép thử sự cố chưa thực hiện.

5.1. Kiểm tra mẫu telemetry từ simulator tới InfluxDB

Simulator phát giá trị cảm biến thay đổi theo thời gian. Năm bản ghi đầu tiên trong bucket iot_raw được đối chiếu với giá trị lưu; sai lệch bằng 0 ở độ chính xác 2 chữ số thập phân. Đây là dữ liệu simulator, chưa phải các trạng thái đặt thủ công trong Wokwi.

Lần	Nhiệt độ telemetry (°C)	Độ ẩm telemetry (%RH)	Khoảng cách telemetry (cm)	Nhiệt độ lưu	Khoảng cách lưu (cm)	Sai lệch (°C / cm)
1	28.10	69.80	100.03	28.10	100.03	0.00 / 0.00
2	27.95	70.27	101.07	27.95	101.07	0.00 / 0.00
3	28.22	69.82	101.26	28.22	101.26	0.00 / 0.00

Lần	Nhiệt độ telemetry (°C)	Độ ẩm telemetry (%RH)	Khoảng cách telemetry (cm)	Nhiệt độ lưu	Khoảng cách lưu (cm)	Sai lệch (°C / cm)
4	28.05	70.78	102.87	28.05	102.87	0.00 / 0.00
5	28.14	70.57	103.54	28.14	103.54	0.00 / 0.00

5.2. Dashboard



Hình 2. Dashboard giám sát cảm biến ESP32 chạy cục bộ với dữ liệu InfluxDB.

5.3. Chất lượng dữ liệu và kiểm thử sự cố

Chỉ tiêu	Giá trị đo được	Nguồn
Gói nhận / ghi thành công	38 điểm sensor_raw trong InfluxDB	Đếm điểm raw; collector ghi từng điểm thành công
Gói lỗi định dạng bị loại	Chưa chốt (không lưu log dừng collector)	stats.invalid không được ghi bền vững
Gói trùng / gói mất	Chưa chốt / 3 gói mất	duplicate chưa chốt; sequence trong InfluxDB
Số lần ESP32 khởi động lại	0	sequence/uptime trong khoảng đo
Outlier (IQR / Z-score)	2 / chưa chạy	log preprocess.py, phương pháp IQR
Số điểm raw → processed	38 → 9	log preprocess.py

Tình huống	Cách tạo	Kết quả theo thiết kế (chưa đo)
Mất Wi-Fi	Đổi SSID trong sketch	Mô phỏng bằng cách bỏ 30 mẫu liên tiếp (60 s). ESP32 không gửi bản tin nên sequence không nhảy và không bị tính là gói mất. Tiền xử lý còn 57/60 cửa sổ 10 s: khoảng trống kéo dài 6 cửa sổ, nội suy được tối đa 3 cửa sổ, 3 cửa sổ còn lại để trống.
Dừng InfluxDB	Dừng influxd.exe bằng Ctrl+C rồi bật lại	Collector giữ 5 điểm trong buffer khi ghi lỗi (ghi nhận 1 lỗi ghi). Sau khi bật lại InfluxDB, hàm flush ghi bù toàn bộ và buffer về 0, không mất điểm nào.
Dừng collector	Ctrl+C rồi chạy lại	sequence nhảy từ 2 lên 33 nên phát hiện 30 gói mất, tương ứng 60 s ở chu kỳ 2 s.
Reset ESP32	Stop rồi Start mô phỏng	sequence về 1 và uptime_s giảm về giá trị nhỏ; collector ghi nhận 1 lần restart, không tính là gói trùng hay gói mất.
Khoảng cách ngoài miền	HC-SR04 trên 400 cm	distance_cm bị bỏ (450 cm vượt miền 2–400 cm); temperature và humidity vẫn được lưu.

6. Phân tích độ trễ, lưu trữ và hiệu năng

6.1. Độ trễ end-to-end

Độ trễ gồm ba chặng: (1) ESP32 → broker → collector, đo bằng latency_ms; (2) ghi InfluxDB, đo bằng write_ms; (3) hiển thị, phụ thuộc chu kỳ làm mới dashboard (mặc định 5 giây) và thời gian truy vấn. Do broker công cộng nằm ngoài mạng nội bộ và Wokwi chạy trên đám mây nên chặng (1) biến động lớn hơn so với mạng LAN, và chặng (3) thường chiếm phần lớn tổng độ trễ.

Chặng	Min (ms)	P50 (ms)	P95 (ms)	Max (ms)
Simulator → collector (latency_ms)	202.0	203.0	203.0	401.0
Ghi InfluxDB (write_ms)	2.0	2.6	10.5	1126.2
Hiển thị trên dashboard	Chưa đo	Chưa đo	Chưa đo	Chưa đo

6.2. Lưu trữ

Với chu kỳ 2 giây, một thiết bị sinh 43.200 bản ghi mỗi ngày, mỗi bản ghi gồm khoảng 7 field. Khi giảm chu kỳ từ 5 giây (Bài 1) xuống 2 giây, lượng dữ liệu tăng 2,5 lần, đúng với phân tích ở câu hỏi 5 của Bài 1 về đánh đổi giữa độ cập nhật và lưu trữ. Resample 10 giây giảm số điểm khoảng 5 lần, vì vậy dữ liệu đã xử lý có thể giữ lâu hơn dữ liệu thô.

Dung lượng thư mục dữ liệu InfluxDB sau thử nghiệm: khoảng 64,2 MiB (gồm cả metadata và dữ liệu cục bộ).

6.3. Đề xuất cải tiến

- Bảo mật: broker công cộng không có xác thực và TLS nên bất kỳ ai biết topic đều đọc hoặc ghi được. Cần chuyển sang HiveMQ Cloud với TLS (cổng 8883), tài khoản riêng, và không để access token trong mã nguồn như sketch Bài 1.

- Dùng QoS 1 hoặc hàng đợi cục bộ trên ESP32 khi mất Wi-Fi; dùng Telegraf thay collector tự viết khi mở rộng nhiều thiết bị.
- Chuyển tiền xử lý thành Task downsampling của InfluxDB; thêm cảnh báo ngưỡng và cảnh báo thiết bị offline dựa trên topic status.
- Edge processing: lọc outlier và lấy trung bình trên ESP32 trước khi gửi để giảm băng thông; với phần cứng thật cần chia áp cho chân ECHO của HC-SR04 (5 V) trước khi vào GPIO 3,3 V.

7. Khó khăn và cách giải quyết

Khó khăn	Cách giải quyết
Wokwi không kết nối được localhost nơi chạy Mosquitto	Dùng broker trên Internet (HiveMQ); collector subscribe cùng broker
Payload dài hơn 256 byte làm publish thất bại	Gọi setBufferSize(512) trên PubSubClient
Topic trên broker công cộng có thể bị người khác ghi đè	Đặt SITE là chuỗi riêng; chuyển sang HiveMQ Cloud có xác thực khi triển khai thật
Gói sau khi ESP32 reset bị nhầm là trùng hoặc mất	Nhận diện restart bằng uptime_s giảm; khóa khử trùng gồm cả ts
HC-SR04 timeout trả -1 làm sai thống kê và outlier	Bỏ khóa distance_cm khi không hợp lệ; collector và tiền xử lý xử lý NaN
IQR cắt cả một số giá trị bình thường ở đuôi phân phối	Dùng Z-score hoặc tăng hệ số k; chỉ nội suy tối đa 3 cửa sổ để không tạo dữ liệu giả
Lộ token trong mã nguồn	Dùng config/.env và .gitignore; đổi access token ThingsBoard đã nằm trong sketch Bài 1

8. Kết luận

Từ nút cảm biến ảo của Bài 1, Bài 2 đã hoàn thiện một pipeline đầy đủ: dữ liệu được thu thập qua MQTT, kiểm tra chất lượng, lưu theo mô hình chuỗi thời gian với retention phù hợp, tiền xử lý và hiển thị gần thời gian thực. Việc tự xây dựng các khâu này làm rõ những vấn đề nền tảng IoT thường che giấu như độ trễ từng chặng, mất gói, trùng gói, khởi động lại thiết bị và bảo mật kênh truyền.

Trong khoảng 80 giây, collector ghi 38 điểm raw; phát hiện 3 gói mất theo sequence. Pipeline IQR tạo 9 điểm processed từ 38 raw và đánh dấu 2 outlier ở distance_cm. Độ trễ latency_ms có P50 203 ms, P95 203 ms; write_ms có P50 2,6 ms, P95 10,5 ms. Mẫu ngắn và dùng simulator nên chưa đại diện cho chạy Wokwi kéo dài. Với chu kỳ 2 giây, tải lý thuyết là 43.200 mẫu/ngày/thiết bị; resample 10 giây còn tối đa khoảng 8.640 cửa sổ/ngày.